

Programiranje za web

Predavanje 5: Oblikovanje API-ja i CRUD domene

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

05.11.2026.



Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

- ➊ Od zapisa do resursa: adrese, operacije i prava
- ➋ Sheme: što ulazi, što izlazi i što se provjerava
- ➌ Popisi: granica odgovora, straničenje i filtriranje
- ➍ Od modela do tablice: migracija, upis i transakcija

Dosad je aplikacija znala tko je korisnik i što smije. Sada dobiva zapise nad kojima radi i sučelje kojim ih netko drugi može stvarati, čitati, mijenjati i brisati.

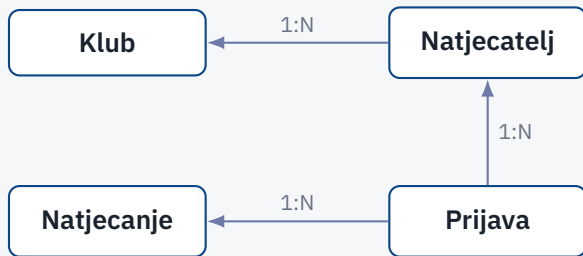
Ishodi učenja

Nakon ovog predavanja student može:

- preslikati četiri osnovne operacije nad zapisom u metode i statusne kodove
- obrazložiti izbor između ugniježdene i samostalne rute
- napisati odvojene sheme za unos, izmjenu i odgovor i reći zašto se razlikuju
- dodati provjeru koja nadilazi tip polja i reći gdje je granica sheme
- ograničiti veličinu odgovora straničenjem i filtriranjem kroz parametre upita
- objasniti zašto se povezani zapisi moraju zatražiti unaprijed
- razlikovati `flush` od `commit` i reći gdje transakcija počinje i završava

Domena primjera

Sustav prijava klubova na powerlifting natjecanje. Uz klub i račun dolaze tri nova zapisa:



Strelica pokazuje na nadređenu stranu: natjecatelj pripada klubu, prijava jednom natjecatelju i jednom natjecanju.

Prijava nije samo spojnica dvaju zapisa. Nosi i vlastite podatke, težinsku kategoriju i najbolji zbroj (`total`), pa je i sama zapis o kojem se vodi računa.

Od zapisa do resursa

Što je resurs, četiri operacije nad njim, pripadnost u adresi, prava koja ruta objavljuje.

Zapis i resurs

Zapis u bazi nema adresu. Postoji kao redak i do njega se dolazi upitom koji zna kako je tablica složena.

Klijent tu bazu nema. Raspolaže samo adresom i metodom, pa mu se zapis mora ponuditi u tom obliku. Zato se svaka vrsta zapisa objavljuje kao **resurs**: nešto što ima svoju adresu i skup radnji koje se nad njom izvode.

U bazi	Prema klijentu
tablica <code>lifters</code>	resurs na <code>/lifters</code>
jedan redak	<code>/lifters/31</code>
stupac <code>first_name</code>	polje u tijelu zahtjeva i odgovora

Preslikavanje ne mora biti jedan na jedan. Resurs smije spojiti podatke iz više tablica, izostaviti stupce koje nitko izvana ne treba, ili uopće ne odgovarati nijednoj tablici.

Četiri operacije nad zapisom

Nad svakim se resursom radi isto: stvara se, čita, mijenja i briše. Te četiri operacije nose naziv CRUD (engl. *create, read, update, delete*) i preslikavaju se u metode i odgovore:

Operacija	Metoda	Kod	Tijelo odgovora
popis	GET	200	niz zapisa
jedan zapis	GET	200	zapis
stvaranje	POST	201	stvoreni zapis
izmjena	PATCH	200	izmijenjeni zapis
brisanje	DELETE	204	nema ga

Stvaranje vraća zapis jer klijent prije odgovora ne zna oznaku koju je baza dodijelila. Brisanje ne vraća ništa, pa uz 204 tijela nema.

Konvencija

Ista tablica vrijedi za svaki resurs, pa tko zna jedan, zna i ostale. To je odluka pri oblikovanju sučelja, a ne zahtjev protokola.

Pripadnost u adresi

Natjecatelj uvijek pripada nekom klubu. Ta se pripadnost u adresi može izraziti na dva načina:

Oblik	Adresa
ugniježđena	<code>/clubs/12/lifters</code>
samostalna	<code>/lifters?club_id=12</code>

U prvom obliku pripadnost je dio adrese, pa je uvijek prisutna i ne može se izostaviti. U drugom je uvjet pretrage: izostavi li se, ruta vraća natjecatelje svih klubova, a to je ishod koji provjera vlasništva mora spriječiti.

Izbor, ne pravilo

Ugniježđeni oblik odgovara odnosu u kojem podređeni zapis bez nadređenoga nema smisla. Samostalni je prikladniji kad se zapis dohvaća po više različitih kriterija, jer bi inače zahtijevao više ugniježđenih adresa za istu vrstu zapisa. U našem su projektu natjecatelji ugniježđeni pod klubom, a natjecanja stoje samostalno.

Gdje se ugnježđivanje zapisuje

Putanje u datoteci s rutama natjecatelja zapisane su bez prefiksa: / za popis i stvaranje, /{lifter_id} za pojedinačni zapis. Puna adresa nastaje tek pri uključivanju u aplikaciju, nadovezivanjem prefiksa na te putanje:

```
# app/main.py
app.include_router(clubs_router, prefix="/clubs")
app.include_router(lifters_router,
                    prefix="/clubs/{club_id}/lifters")
app.include_router(competitions_router, prefix="/competitions")
app.include_router(registrations_router,
                    prefix="/competitions/{comp_id}/registrations")
```

Time je oblik sučelja vidljiv na jednom mjestu. Iz ovog se popisa čita koja je pripadnost ugniježđena, bez otvaranja ijedne datoteke s rutama.

Oznaka {club_id} iz prefiksa stiže u funkciju rute kao običan argument, pa je ne treba nigdje posebno izvlačiti.

Čitanje i pisanje ne traže isto

Natjecanja čita svatko tko je prijavljen, a mijenja ih samo administrator. Ta se razlika ne vidi u tijelu funkcije nego među njezinim ovisnostima:

```
# app/routers/competitions.py
@router.get("/", response_model=list[CompetitionResponse])
async def list_competitions(
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    return await competition_service.list_competitions(db)

@router.post("/", response_model=CompetitionResponse, status_code=201)
async def create_competition(
    body: CompetitionCreate,
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_role("admin")),
):
    return await competition_service.create_competition(db, body)
```

Razlika je u jednoj ovisnosti. `require_role` traži određenu ulogu, a `get_current_user` samo to da je korisnik utvrđen, bez obzira na to koja mu je uloga.

Brisanje i povezani zapisi

Obriše li se natjecatelj, prijave koje na njega pokazuju ostaju bez zapisa na koji se odnose. Tri su izlaza iz toga:

Izlaz	Posljedica	U projektu
zabraniti dok veze postoje obrisati i povezane zapise	zahtjev se odbija, zapis ostaje nestaje i prijava i povijest o njoj	natjecatelj —
označiti stanje umjesto brisanja	zapis ostaje, oznaka kaže da ne vrijedi	prijava

Izbor ovisi o tome treba li povijest zapisa ostati dostupna. Ono što se ne smije dogoditi jest da izbora nema: ruta koja briše bez ijednog od tih pravila završava neuhvaćenom pogreškom baze umjesto odgovorom koji klijent razumije.

Brisanje natjecatelja koji ima prijave vraća 409 i kod `has_registrations`.

Scheme

Što shema provjerava, zašto jedna nije dovoljna, unos i odgovor, izmjena i neposlana polja, idempotentnost, provjere koje nadilaze tip.

Shema kao opis ulaza i izlaza

Shema je zapis o tome kakve podatke ruta prima i kakve vraća. Framework je provjerava prije nego zahtjev dođe do poslovnih pravila, pa neispravan sadržaj do njih ne stiže.

Shema provjerava:

- jesu li obavezna polja prisutna
- je li svako polje očekivanog tipa
- zadovoljava li vrijednost ograničenje zapisano uz polje, primjerice popis dopuštenih vrijednosti ili raspon

Shema: primjer

```
# app/schemas/lifter.py
class LifterCreate(BaseModel):
    first_name: str = Field(min_length=1, max_length=80)
    birth_date: date
    gender: Literal["M", "F"]
```

Poslani spol

Ishod

„M”, vrijednost s popisa

„X”, nije na popisu

3

prolazi dalje

odbijeno, odgovor imenuje polje gender

odbijeno, vrijednost nije niz znakova

`Literal` nabraja dopuštene vrijednosti, pa provjera stoji u samoj deklaraciji polja.

Isti zapis, različit skup polja

Natjecatelj je u bazi jedan zapis, ali za svaku vrstu zahtjeva putuje drukčiji skup polja. Polje koje se već nalazi u adresi rute u tijelu se ne ponavlja:

Polje	Stvaranje	Izmjena	Odgovor
id	nema ga	u adresi	uvijek
ime i prezime	obavezno	ako se mijenja	uvijek
datum rođenja	obavezno	ako se mijenja	uvijek
spol	obavezno	ako se mijenja	uvijek
club_id	u adresi	u adresi	uvijek

Stvaranje i izmjena traže različito, a odgovor nosi sve, uključujući i ono što klijent nikada ne šalje: id koji je dodijelila baza i club_id koji je stajao u adresi.

Koliko nam je shema potrebno?

Jedna shema ne bi mogla ispuniti sve tri uloge, jer bi ista polja morala istodobno tražiti i ne tražiti. Zato ih razdvajamo na `LifterCreate`, `LifterUpdate` i `LifterResponse`.

Tri sheme nisu pravilo. Natjecatelj se pojavljuje u trima ulogama, pa ih ima tri; drugi zapisi mogu imati različit broj:

- **Prijava ima četvrtu.** `LifterBrief` sadrži samo ime i prezime i ugrađuje se u odgovor o prijavi.
- **Autentikacija ima drukčiji skup shema.** Tamo stoje `LoginRequest`, `TokenResponse` i `RefreshRequest`, jer se ne stvara zapis nego izdaje token.

Konvencija

Podjela na unos, izmjenu i odgovor nije propisana nego je uobičajen način da se te tri uloge razdvoje. Broj shema u svakom slučaju slijedi iz toga u koliko se različitih oblika isti zapis pojavljuje u zahtjevima i odgovorima.

Schema za unos i odgovor

```
# app/schemas/lifter.py
class LifterCreate(BaseModel):
    first_name: str = Field(min_length=1, max_length=80)
    last_name: str = Field(min_length=1, max_length=80)
    birth_date: date
    gender: Literal["M", "F"]

class LifterResponse(BaseModel):
    id: int
    first_name: str
    last_name: str
    birth_date: date
    gender: str
    club_id: int

model_config = {"from_attributes": True}
```

Ulazna shema nema `id` ni `club_id`: prvi dodjeljuje baza, a drugi je već u adresi rute. Postavka `from_attributes` dopušta da se izlazna shema popuni izravno iz zapisa koji je vratio ORM.

Izlazna shema nije samo opis. Odgovor prolazi kroz nju, pa polje koje u njoj ne stoji ne može izaći ni greškom.

Izmjena i neposlana polja (I)

```
# app/schemas/lifter.py
class LifterUpdate(BaseModel):
    first_name: str | None = Field(default=None, min_length=1, max_length=80)
    last_name: str | None = Field(default=None, min_length=1, max_length=80)
    birth_date: date | None = None
    gender: str | None = Field(default=None, pattern=r"^[MF]$")
```

Sva su polja neobavezna, pa klijent šalje ona koja mijenja. Time nastaje dvoznačnost: izostalo polje i polje poslano kao null u shemi izgledaju jednako, a znače različito. Posljedica je da bi izmjena jednoga polja obrisala sva ostala.

Izmjena i neposlana polja (II)

```
# app/services/lifter_service.py
for field, value in body.model_dump(exclude_unset=True).items():
    setattr(lifter, field, value)
```

`exclude_unset` razdvaja ta dva slučaja: u rječnik ulaze samo polja koja su doista poslana. Prazno tijelo shema propušta, pa zapis ostaje nepromijenjen, a odgovor je i dalje 200.

Izmjena i neposlana polja (III)

`exclude_unset` razlikuje poslano od neposlanoga, ali ne i poslano kao `null`. Takvo je polje poslano, pa ulazi u rječnik i pokušava upisati `null` u stupac koji ga ne dopušta. Odgovor tada dolazi iz baze, a ne iz sheme:

```
# app/schemas/common.py
def reject_null(value):
    if value is None:
        raise ValueError("Polje ne smije biti null; izostavite ga")
    return value

# app/schemas/lifter.py
_not_null = field_validator(
    "first_name", "last_name", "birth_date", "gender", mode="before"
)(reject_null)
```

Validator se poziva samo za poslana polja, pa izostalo polje i dalje prolazi. Zahtjev s `null` dobiva 422 i ime polja, kao i svaka druga pogreška u obliku podataka.

Idempotentnost

Metoda je idempotentna ako niz od više jednakih zahtjeva ostavlja poslužitelj u istom stanju kao i jedan takav zahtjev. Standard to svojstvo pridružuje samim metodama:

- GET, HEAD, OPTIONS i TRACE ne mijenjaju stanje, pa su idempotentni.
- PUT i DELETE mijenjaju stanje, ali su idempotentni: zapis se postavlja na zadanu vrijednost, odnosno briše.
- POST i PATCH nisu idempotentni.

Posljedica je praktična: klijent smije ponoviti neuspjeli GET ili DELETE bez razmišljanja. Ponovljeni POST može stvoriti drugi zapis, pa zaštita od toga mora doći iz aplikacije, najčešće ograničenjem jedinstvenosti u bazi.

PUT i PATCH

Obje metode mijenjaju postojeći zapis. Razlikuju se po tome što klijent šalje i što se događa s onim što nije poslao:

	PUT	PATCH
Klijent šalje	cijeli zapis	izmijenjena polja
Izostavljeno polje	briše se	ne mijenja se
Ponovljeni zahtjev	isto stanje	ne nužno isto stanje
Stvaranje pod zadanom oznakom	moguće	nije moguće
Obrada u aplikaciji	zamjena	spajanje s postojećim

Treći redak je idempotentnost: PUT se smije ponoviti nakon prekinute veze, jer ishod ne ovisi o tome je li prvi zahtjev stigao. Kod PATCH-a to općenito ne vrijedi.

Odabir metode za izmjenu

PUT odgovara kad klijent drži cjelovit prikaz zapisa i mijenja ga u cijelosti, primjerice postavke ili dokument. Tada je brisanje polja izražen postupak: polje koje klijent više ne pošalje nestaje. Obrada je jednostavnija, jer nema spajanja ni dvoznačnosti između neposlanog polja i poslanog `null`.

PATCH odgovara kad je zapis velik, a izmjena malena, i kad klijent uopće ne poznaje sva polja. Zamjena bi tada izbrisala sve čega se klijent ne sjeća poslati, pa je slanje samo izmijenjenog i sigurnije i jeftinije.

Izbor, ne pravilo

U našem projektu izmjena je svugdje izvedena metodom PATCH, jer klub mijenja jedno ili dva polja natjecatelja i ne drži cijeli zapis. Sučelje koje se odluči za PUT ništa ne krši, ali mora računati na to da izostavljeno polje znači brisanje.

Provjere koje nadilaze tip

Tip polja hvata pogrešan oblik, ali ne i besmisleni vrijednost. Datum rođenja u budućnosti je ispravan datum:

```
# app/schemas/lifter.py
class LifterCreate(BaseModel):
    birth_date: date

    @field_validator("birth_date")
    @classmethod
    def must_be_past(cls, v: date) -> date:
        if v >= date.today():
            raise ValueError("Datum rođenja mora biti u prošlosti")
        return v
```

Kategorija prijave traži istu vrstu provjere. Težinske kategorije nisu cijeli brojevi, jer uz "83" i "93" postoje i otvorene kategorije "120+" i "84+", pa je polje niz znakova. Popis dopuštenih vrijednosti onda provjerava validator, koji poslanu vrijednost imenuje u poruci ako u popisu nije.

Kad takva provjera padne, odgovor ima isti oblik kao i pogreška u tipu: imenuje polje i navodi razlog. Sloj s poslovnim pravilima time nikada ne prima vrijednost koja nema smisla.

Dokle shema seže

Shema vidi poslani sadržaj i ono što proces sam zna, primjerice današnji datum. Može usporediti i dva polja istog zahtjeva:

```
# app/schemas/competition.py
@field_validator("final_deadline")
@classmethod
def final_after_prelim(cls, v, info):
    prelim = info.data.get("prelim_deadline")
    if prelim and v <= prelim:
        raise ValueError("final_deadline mora biti nakon prelim_deadline")
    return v
```

Rokovi su u shemi tipa `AwareDatetime`, pa zahtjev bez oznake zone ne prolazi. Ono što ne može jest pogledati u bazu. Validator je obična funkcija bez sesije, a u asinkronom radu unutar nje nema ni čekanja odgovora baze. Granica je time jasna: shema prosuđuje vrijednost samu za sebe, ne u odnosu na druge zapise. Je li natjecatelj već prijavljen, bez baze se ne zna, pa je to posao sloja `services`, koji dolazi iza nje.

Odgovor koji nosi povezani zapis

Popis prijava u kojem stoji samo `lifter_id` klijentu ne znači mnogo; za svako ime trebao bi novi zahtjev. Zato odgovor nosi i sam povezani zapis, ali suženo:

```
# app/schemas/registration.py
class LifterBrief(BaseModel):
    first_name: str
    last_name: str

    model_config = {"from_attributes": True}

class RegistrationResponse(BaseModel):
    id: int
    lifter_id: int
    competition_id: int
    category: str
    total: int
    status: str
    lifter: LifterBrief
```

`LifterBrief` sadrži samo ime i prezime. Datum rođenja i klub u popisu prijava nikome ne trebaju, a shema odgovora je mjesto na kojem se odlučuje koja polja povezanog zapisa izlaze van, a koja ostaju u aplikaciji.

Popisi

Zašto popis ne vraća sve, straničenje i filtriranje, redoslijed, povezani zapisi u popisu.

Popis ima granicu

Pri oblikovanju rute koja vraća popis treba odlučiti koliko zapisa ona smije vratiti. Odluka „sve” zadovoljava dok ih je malo, a s rastom broja zapisa ima tri posljedice: upit čita cijelu tablicu, odgovor raste, a klijent prima više nego što može prikazati. Zato granica ulazi odmah:

```
# app/routers/lifters.py
@router.get("/", response_model=list[LifterResponse])
async def list_lifters(
    club_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
    gender: str | None = Query(default=None, pattern=r"^[MF]$"),
    limit: int = Query(default=20, le=100),
    offset: int = Query(default=0, ge=0),
):
    ...
```

limit kaže koliko zapisa, offset koliko ih preskočiti. Vraćanje popisa u dijelovima naziva se straničenje (engl. *pagination*).

Zadane vrijednosti i gornja granica

- **Klijent koji o straničenju ne zna ništa dobio bi neograničen odgovor.** Zato obje vrijednosti imaju zadanu, pa se ruta ne može ni slučajno pozvati bez granice.
- **Klijent može zatražiti milijun zapisa.** Zato `limit` ima gornju granicu `1e=100`, inače se ruta vraća na ponašanje s početka.
- **Neispravna vrijednost filtra došla bi do upita.** Zato `gender` ima uzorak `^[MF]$,` koji je odbija prije toga.

Parametri upita opisani su jednako strogo kao i polja sheme, pa ih framework odbija prije nego funkcija počne raditi.

Konvencija

Nazivi `limit` i `offset` nisu propisani. U drugim se sučeljima sreću i `page` i `per_page`, a neka uz popis vraćaju i ukupan broj zapisa. Izbor je slobodan, ali mora biti isti na svim rutama istoga sučelja.

Filtar i granica u upitu

```
# app/repositories/lifter_repo.py
stmt = select(Lifter).where(Lifter.club_id == club_id)
if gender is not None:
    stmt = stmt.where(Lifter.gender == gender)
stmt = stmt.order_by(Lifter.id).limit(limit).offset(offset)
```

Upit se gradi postupno: pripadnost klubu provjerava se uvijek, a spol samo ako je filtar poslan. Ruta i servis samo prosljeđuju vrijednosti, a `select` i nazivi stupaca ne izlaze iz sloja `repositories`. Filtar i granica tako djeluju na upit, a ne na već dohvaćeni popis.

Straničenje ne može ispravno raditi bez izrijeком zadanog poretka zapisa, jer ga baza sama ne jamči. Bez `order_by` postoji mogućnost da jedan zapis dođe i na prvoj i na drugoj stranici, dok drugi ne dođe ni na jednu.

Konvencija

Uz `offset` baza pri svakom zahtjevu preskače sve prethodne retke, pa duboke stranice postaju spore. Drugi pristup pamti oznaku zadnjeg viđenog zapisa i traži one iza nje. Brži je i otporan na umetanje, ali ne dopušta skok na proizvoljnu stranicu. U našem se projektu koristi `offset`, jer su popisi kratki.

Lijeno učitavanje povezanog zapisa

RegistrationResponse ima polje `lifter`, pa odgovor o prijavi nosi i ime natjecatelja. Do tog se zapisa dolazi preko veze opisane u modelu:

```
# app/models/registration.py
lifter: Mapped[Lifter] = relationship(back_populates="registrations")
competition: Mapped[Competition] = relationship(
    back_populates="registrations"
)
```

Ta dva retka nisu stupci tablice nego veze. Zahvaljujući njima povezani se zapis čita kao svojstvo, dakle `registration.lifter`, bez pisanja upita. ORM taj zapis ne dohvaća unaprijed nego tek u trenutku čitanja svojstva. Taj se postupak naziva lijeno učitavanje (engl. *lazy loading*).

Lijeno učitavanje u asinkronom radu

U sinkronom radu lijeno učitavanje prolazi neopaženo: dretva pošalje upit, pričekava odgovor i nastavi. Čekanje je dio samog čitanja svojstva i nigdje se ne vidi.

U asinkronom radu, za koji smo se opredijelili pri postavljanju veze s bazom, to nije moguće. Čekanje odgovora baze mora biti izrijeком označeno s `await`, a čitanje svojstva nije izraz uz koji `await` može stajati.

Posljedica je da `registration.lifter` ne vrati podatke nego podigne iznimku. Pritom se to ne dogodi pri dohvat prijava nego tek kasnije, u trenutku kad se shema odgovora pokušava popuniti, pa poruka o pogrešci upućuje na mjesto udaljeno od uzroka.

Dohvat povezanih zapisa unaprijed

Lijeno učitavanje ima i drugu cijenu: popis od pedeset prijava izveo bi pedeset i jedan upit, jedan za sam popis i po jedan za natjecatelja svake prijave. Toliki se broj upita naziva problemom $N+1$. Za vrijeme razvoja aplikacije, kad je broj prijava mali, razlika se ne primijeti, ali u produkciji s nekoliko tisuća zapisa postaje najskuplji dio odgovora.

Oba se rješavaju istim zahvatom, traženjem povezanih zapisa u istom upitu, dok se za odgovor baze još smije čekati:

```
# app/repositories/registration_repo.py
stmt = (
    select(Registration)
    .options(selectinload(Registration.lifter))
    .where(Registration.competition_id == comp_id)
)
```

`selectinload` nalaže ORM-u da nakon upita za prijave izvede još jedan upit, kojim odjednom dohvaća sve njihove natjecatelje. Dakle dva upita ukupno, bez obzira na broj prijava, i nijedno čitanje svojstva koje bi podiglo iznimku.

Od modela do tablice

Zapis opisan u kodu, uvođenje nove tablice, upis i potvrda transakcije.

Zapis opisan u kodu

Svaki resurs ima svoj model. Prijava je među njima najpotpunija, jer nosi i vezu prema drugim zapisima i vlastite podatke:

```
# app/models/registration.py
class Registration(Base):
    __tablename__ = "registrations"

    id: Mapped[int] = mapped_column(primary_key=True)
    lifter_id: Mapped[int] = mapped_column(ForeignKey("lifters.id"))
    competition_id: Mapped[int] = mapped_column(
        ForeignKey("competitions.id")
    )
    category: Mapped[str] = mapped_column(String(20), nullable=False)
    total: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    registered_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
```

Dva strana ključa nose vezu, category i total vlastite podatke, a registered_at popunjava baza, pa vrijeme upisa ne ovisi o satu poslužitelja.

Uvođenje nove tablice

Model opisan u Pythonu još nije tablica. Alembic uspoređuje stanje baze s onim što vidi u `Base.metadata`, a tamo dolaze samo razredi koji su negdje uvezeni:

```
# app/models/__init__.py
from app.core.database import Base
from app.models.club import Club
from app.models.competition import Competition
from app.models.lifter import Lifter
from app.models.registration import Registration
from app.models.user import User
```

Izostane li redak, `alembic revision --autogenerate` neće prijaviti pogrešku. Napraviti će praznu migraciju, jer razliku između modela i baze nije ni vidio.

Nova tablica tako ne nastaje pokretanjem aplikacije nego migracijom, jednako kao i prva tablica u projektu.

Tri koraka upisa

```
# app/repositories/lifter_repo.py
async def create(db: AsyncSession, lifter: Lifter) -> Lifter:
    db.add(lifter)
    await db.flush()
    return lifter
```

- `db.add` stavlja objekt u sesiju. U bazi se još ništa nije dogodilo i zapis nema oznaku.
- `db.flush` šalje naredbu upisa, unutar otvorene transakcije. Baza dodjeljuje oznaku i zapis je od tog trenutka vidljiv daljnjim upitima iste transakcije.
- Zapis još nije trajan. Ako se transakcija poništi, nestaje, a do potvrde ga nijedan drugi zahtjev ne vidi.

Zato servis smije odmah nakon `create` dohvatiti isti zapis i pročitati njegovu oznaku, iako transakcija još nije potvrđena.

Tko potvrđuje transakciju

Kad bi sloj `repositories` sam potvrđivao, svaki bi upis postao trajan čim se izvede. Operacija koja upisuje dva zapisa pa padne na drugome ostavila bi prvi u bazi.

```
# app/core/deps.py
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

Zato sloj `repositories` ide samo do `flush`, a `commit` izvodi ovisnost koja drži sesiju, i to tek kad cijela obrada prođe bez iznimke. Izađe li iznimka iz bilo kojeg sloja, slijedi `rollback` i u bazi ne ostaje ništa od tog zahtjeva.

Sažetak

- Resurs je vrsta zapisa objavljena pod vlastitom adresom; preslikavanje iz tablice ne mora biti jedan na jedan.
- Nad svakim se resursom ponavljaju iste četiri operacije; stvaranje vraća zapis jer oznaku dodjeljuje baza, brisanje ne vraća ništa.
- Ugniježdjena adresa nosi pripadnost i ne može se izostaviti; kod samostalne je pripadnost uvjet pretrage i mora se provjeriti kao i svaki drugi.
- Unos, izmjena i odgovor tri su različite sheme, a `exclude_unset` razlikuje neposlano polje od poslanog praznog.
- Shema prosuđuje vrijednost samu za sebe; pravilo koje traži uvid u bazu pripada sloju iznad nje.
- Popis se vraća u dijelovima, s gornjom granicom i zadanim redoslijedom; povezani zapisi traže se unaprijed.
- `flush` šalje upis i smije stajati u sloju `repositories`; `commit` ga čini trajnim i izvodi ga ovisnost koja drži sesiju.